

Graphics Final Project: Image to WaterColor

Background Information:

For background research, we utilized a video that explained high level ideas on what's necessary to make an image have a water color effect, this included ideas like edge darkening, non uniform fill, irregular borders, layered transparency. Edge darkening when edges of a shape dry darker than the center. Non uniform fill is where paint does not cover an area evenly, and this can happen due to textured paper, and we want that finish on our final product. Irregular borders happen due to wet on wet techniques. Lastly we want layered transparency and this is the ability to see through layers of paint. This is especially an important aspect of our project because we plan to start with a “wet image” as a base and want to add on to that.

What we've Accomplished:

Our final result is this: when the user clicks the mouse, a water drop is dropped and “mixes” with the underlying images, leaving a watercolor effect. The water drop spreads how water would and if the user clicks and drags the mouse, a continuous stream of water is dropped.

Watercolor Effect Code Explained:

Irregular Borders:

We implemented irregular borders by adding multiple layers of noise with a simple sin function and then using fbm, aka fractal brownian motion, which essentially adds extra layers to the noise to give a non uniform effect.

Layer Transparency:

To implement layered transparency, we implemented it by showing how paint can build up into multiple bands of layers where they are translucent, and we did this by having a loop where each iteration shows one level of brightness. We have 6 layers and we would sample a blurred version of the image. The whole layered result created using mix was blended against the normal color. We made sure there weren't hard layers but soft blended ones by adding some padding.

Blur:

For the blur we wrote a blurSample helper that takes 5 samples of the texture one at the center and four at the corners offset by radius r and averages them. It's enough to soften the image into the watercolor look we wanted. The radius scales with combined, our wetness value ($r = \text{offset} * (1.0 + \text{combined} * 2.5)$), so areas with more "water" blur more harshly while dry areas stay sharper.

Edge Darkening:

For edge darkening we created a drop radius and drop center variable so we can easily have the darkness be on the edges, when we were implementing the entire image to be watercolor, the drop radius was set to be a big number and the center was the center of the image. This was done this way so it will be easy to update the code to work with mouse controls. For edge darkening with the whole image, we also created helper functions to create that vignette effect. When we implemented mouse controls, the logic of edge darkening switched a bit to where we have the darker ring be created at the edge boundary. We have variables like ringInner, ringStrength, and edgeness to form the ring, and we played around with the values until we got the effect we wanted.

Mouse Code:

We attached four listeners to the 2D canvas: mousedown flips an isPainting flag and drops paint immediately, mousemove keeps painting while the flag is set, and mouseup/mouseleave flip it off and clear lastDrop so the next click starts a fresh stroke. The handler addDrop takes the raw mouse event and converts it to normalized [0,1] coordinates using getBoundingClientRect so the math doesn't break if the canvas gets resized.

The trickiest part was making drags feel continuous. If we only painted once per mousemove event, fast drags came out as a string of disconnected dots because the browser doesn't fire an event at every pixel. So we track lastDrop and, when it exists, compute the distance to the new position and stamp paint along the line at intervals of about a quarter brush-radius. That gives a solid stroke regardless of drag speed.

Drop Diffusion:

For this, we passed two variables from [App.ts](#) to [Shader.ts](#): u_time and u_dropCenter. We keep track of time in [App.ts](#) using startTime. We also have a variable called dropCenter and drops to keep track of all the drops (so clicking the mouse doesn't erase previous drops). We made two functions, updateDrops and paintSoft, to ensure the water drops spread correctly and with the correct amount of clearness. In the shader, there's a fair amount of math to calculate how far the water spreads, for how long, and the colors each pixel should be colored (based on the watercolor effect).

Artifacts produced:

We worked on this project by implementing it in a way where it makes changes to the whole image and then we worked on mouse control and the water drops. Then we changed our code so it works for the pixels the water drops touch, rather than changing all the pixels at once. Here are pictures of our final outcome.









References

Gamelogic. “ Turn 3D into Watercolor Magic: Step-By-Step Shader Tutorial .YouTube, 12 May 2025, www.youtube.com/watch?v=YMp7VaXuB5A. Accessed 30 Apr. 2026.